

❖❖ Entrega 02: Casos de Uso, API, Persistencia y Seguridad

Curso: Programación Avanzada

Unidad: Arquitecturas Empresariales y Materialización de APIs RESTful

Tiempo estimado: Semanas 6 a 11

Equipo: Máximo 3 integrantes

1. Objetivo de la Entrega

Completar la materialización del **Sistema de Triage y Gestión de Solicitudes Académicas** construyendo un Backend 100% funcional, seguro y comunicado mediante red. Los estudiantes deben demostrar su capacidad para:

Implementar la lógica orquestadora mediante **Casos de Uso** desacoplados. Construir y exponer una **API RESTful** madura basada en los contratos del Dominio. Garantizar la integridad e inmutabilidad de los datos en tránsito utilizando **Records** para los **Data Transfer Objects (DTOs)**.

Materializar la persistencia de datos en una base de datos relacional híbrida H2 utilizando **JPA/Hibernate** y **Spring Data**.

Proteger el ecosistema de la API mediante **Autenticación y Autorización con JWT** (HMAC/HS256) sobre Spring Security.

Esta entrega consolida el desarrollo backend estructurado en capas (Arquitectura Hexagonal/Limpia), agrupando orgánicamente todo el conocimiento impartido entre las guías 06 a 12.

2. Alcance Funcional

En esta iteración se deberán exponer y persistir todos los Requerimientos Funcionales (RF) contemplados previamente en la fase de diseño.

RF	Descripción	Cobertura
RF-01	Registro de solicitudes	Backend Expuesto
RF-02	Clasificación de solicitudes	Backend Expuesto
RF-03	Priorización de solicitudes	Backend Expuesto
RF-04	Gestión del ciclo de vida (estados)	Backend Expuesto
RF-05	Asignación operativa de responsables	Backend Expuesto
RF-06	Historial auditable	Base de Datos + Endpoint
RF-07	Consultas, filtrado y paginación	Consultas Dinámicas
RF-08	Cierre de solicitudes (reglas)	Reglas Expuestas REST
RF-13	Definición de roles (Autenticación JWT)	Backend Seguro

❔❔ Los requerimientos asociados al Frontend (interfaces visuales completas) o despliegues productivos quedan para posibles entregas posteriores. El testing automatizado (Guía 13) no forma parte del peso evaluativo en esta entrega, priorizando la estructura y acoplamiento limpio entre componentes.

3. Entregables

El proyecto Spring Boot creado en la *Entrega 01* deberá reestructurarse/expandirse para contener los siguientes entregables:

A. Capa de Aplicación (application)

1. **Casos de Uso (UseCases)**: Clases orquestadoras que interactúen con el Dominio y no reciban implementaciones de framework (solo interfaces/puertos).
2. **Puertos de Persistencia**: Interfaces de abstracción que dictan cómo buscar o almacenar datos (ej. `SolicitudRepository` o `UsuarioRepository`) operando estrictamente con entidades base del Dominio.

B. Capa de Infraestructura - Presentación API REST

(infrastructure.rest)

1. **Controladores (@RestController)**: Sirven como el principal punto de anclaje para los verbos HTTP mapeados desde Swagger/OpenAPI.
2. **DTOs inmutables (Records)**: Consumo 100% migrado a record de Java tanto para peticiones (Requests) como para respuestas (Responses) con validación mediante `jakarta.validation`.
3. **Mappers Controlables**: Mecanismo de traducción explícito (ej. *MapStruct* o *Hand-Mapped*) que conviertan de DTO a Dominio y de Dominio a DTO.
4. **Manejo Excepcional Global**: Clase `@ControllerAdvice` (`GlobalExceptionHandler`) que procese asertivamente las excepciones del negocio a códigos estándar (HTTP 400 Bad Request , 404 Not Found).

C. Capa de Infraestructura - Persistencia JPA (infrastructure.persistence.jpa)

1. **Entidades JPA (@Entity)**: Materialización estructural (columnas, tablas, relaciones uno-a-uno/uno-a-muchos) separadas semánticamente de las entidades de Dominio.
2. **Value Objects incrustables (@Embeddable)**: Para propiedades profundas como eventos históricos.
3. **Spring Data Repositories**: Interfaces autogeneradas por JPA para interactuar con la Base de Datos H2 (`JpaRepository`).
4. **Adapter de Persistencia**: La clase que verdaderamente implementa el Puerto que clama nuestra Capa de Aplicación, actuando de puente hacia la DB mediante un Mapper.

D. Capa de Infraestructura - Seguridad JWT (infrastructure.security)

1. **Cerradura Estática HMAC**: Propiedad genérica configurada correctamente en `application.properties` (al menos 256 bits).
2. **Configuración Lambda Declarativa (SecurityFilterChain)**: Protegiendo rutas de dominio y permitiendo acceso libre a `/auth/login` , endpoints de swagger y endpoints web H2 (`/h2-console`).
3. **Generación de JWT y Servicios Auth**: Controladores para autenticación que expidan sus respectivos Tokens verificados por BCrypt en H2.
4. **Seed de Datos (CommandLineRunner)**: Configuración estática opcional que ingrese administradores de prueba (Agentes/Coordinadores) la primera vez que se ejecute la base de datos relacional.

4. Formato de Entrega

1. **Repositorio en GitHub** — El mismo de la iteración pasada o una rama/fork consolidado.
2. **README.md** actualizado con:
 - Recuento de endpoints habilitados, o credenciales *Seed* predefinidas.
 - Instrucciones concisas para levantar la base de datos H2 en consola remota.
3. **Directorio /docs** con:
 - Archivo de documentación de Endpoints (`openapi.yaml`).
 - (Opcional): Una colección JSON de de llamadas HTTP extraída desde herramientas como **Postman**, útil para probar validaciones de roles, flujos correctos e inválidos.
4. **Código fuente en `src/main/java`** con los paquetes expandidos en un modelo completamente Hexagonal o de Capas Limpias (`domain` , `application` , `infrastructure`).

5. Criterios de Evaluación (Rúbrica)

Criterio	Peso	Descripción
Arquitectura de Software y Capas	20%	El código está limpiamente dividido. La capa de negocio jamás importa <code>jakarta.persistence</code> , <code>spring-web</code> , ni dependencias ajenas. Los flujos cruzan correctamente de Infraestructura -> Aplicación -> Dominio.
Controladores, Rutas REST y DTOs	25%	Los endpoints mapean semánticamente a REST. Existe uso correcto de verbos (POST, PUT, GET, PATCH). Los DTOs hacen uso explícito de <code>record</code> de Java . Se validan asertivamente los errores HTTP desde el interceptor de excepciones.
Mapeos de Datos e Hibridación JPA	25%	Existe una frontera clara entre las Entidades de la base de datos (<code>@Entity</code>) y las Entidades puras del Dominio; además, las consultas transaccionales, filtrados y paginaciones (Guía 11) operan de forma estable sobre la base de datos H2 .

Criterio	Peso	Descripción
Filtros de Seguridad Web (JWT)	20%	Las variables se inyectan correctamente gracias a un secret compartido HS256 . Los filtros protegen adecuadamente áreas sensibles. El sistema logra expedir contraseñas tokenizadas verificando exitosamente el inicio de sesión.
Calidad y Documentación	10%	Presencia de la especificación openapi.yaml , uso de nombres explícitos e identificables basados en el Lenguaje Ubicuo y el Repositorio de código mantiene orden profesional en los commits.

Nota Crítica Institucional: Toda calificación estará supeditada a sustentación del proyecto validando la apropiación del equipo en tópicos base como Spring Security, Repositorios Mapeados o inmutabilidad (Java Records). El grado de defensa dominará la métrica final y se multiplicará por la nota aquí descrita.

6. Trazabilidad: Guías → Entregables

Guía	Temática Desarrollada	Se refleja y evalúa en
06 — Casos de Uso	Comportamiento, Inversión de dependencias	Capa application/usecases
07 — Diseño API / Dominio	Verbos HTTP, Records, Códigos HTTP	Open API YAML (/docs)
08 — Controllers REST	Interceptores, REST, Data Transfer Objects	DTOs (Records), ControllerAdvice, REST
09 — Materialización de Flujos	Integración de Capas (DI)	Arquitectura final de carpetas y llamadas correctas (Mappers)
10 — Persistencia H2 y Repos	Adaptadores @Entity , JPA, SQL	Capa infrastructure/persistence

11 — Consultas Estructuradas	Paginación y Filtrados dinámicos	JPQL, Endpoints de Listados paramétricos
-------------------------------------	----------------------------------	--

Guía	Temática Desarrollada	Se refleja y evalúa en
12 — Seguridad Integral	JWT, Roles, Authentication Managers	Interceptor HttpSecurity, AuthController, Hash BCrypt